多模态大模型系列报告

会用工具的眼睛：多模态智能体与工具使用

当模型从”回答问题”走向”改变世界”——推理-行动循环、工具学习与 computer-use 智能体的完整谱系

多模态大模型系列报告

2026-09-29

本报告为系列技术综述，内容基于公开论文整理，仅供学习研究使用。

目录

1 引言：本方向在多模态版图中的位置 4

2 基础知识 5

2.1 从一次性映射到决策循环：agent 的形式化 5
2.2 思维链与” 用语言表达中间状态” 6
2.3 动作空间设计：agent 的” 手” 长什么样 6
2.4 工具调用的三个子问题 7
2.5 Grounding：从语言决策到物理动作 7
2.6 Agent 的评测：为什么” 打分” 本身是个研究问题 8
2.7 RAG：最朴素的工具使用 8

3 经典方法谱系 9

3.1 推理-行动框架：从 ReAct 到 Reflexion 到搜索 9
3.2 工具学习：从教模型调 API 到万级 API 生态 9
3.3 Web agent：从静态数据集到真实互联网 10
3.4 GUI / computer-use agent：从外挂管线到原生模型 10
3.5 关键论文一览表 10

4 代表作深度剖析 13

4.1 ReAct：推理与行动的联姻（2022） 13
4.2 ToolLLM：把工具学习推进到万级 API（2023） 14
4.3 WebArena：给 web agent 一个真实的考场（2023） 15
4.4 OSWorld：把考场搬进真实操作系统（2024） 15
4.5 UI-TARS：原生 GUI agent 模型的崛起（2025） 16

5 研究前沿（2025–2026） 17

5.1 RL post-training：GUI agent 的新主战场 17
5.2 组合架构：generalist 与 specialist 的分工 18
5.3 MCP：工具接口的标准化与生态 18
5.4 Agentic RAG 与多模态检索 18
5.5 记忆与长程规划：从外挂模块到自我进化 19
5.6 评测的下一程：更长、更真、更安全 19

6 技术路线对比与权衡 **19**

6.1 观测表示：截图、DOM，还是两者？ 19

6.2 工具获取方式：提示、微调，还是强化学习？ 20

6.3 系统形态：单一原生模型 vs 多模块框架 21

6.4 评测维度权衡 21

7 开放问题与未来方向 **22**

8 研究机会点评（给入门研究者的建议） **22**

9 术语表 **23**

10 参考文献 **24**

摘要 多模态智能体（multimodal agent）是多模态大模型版图中”离真实世界最近”的方向：它不再满足于看图说话，而是要求模型在网页、桌面、手机等真实数字环境中感知、决策、行动，并借助外部工具弥补自身能力边界。本报告以”模型如何学会使用工具、操作界面”为主线，系统梳理 2022–2026 年该方向的完整演化。我们从范式原点讲起：ReAct 把思维链（chain-of-thought）与行动（action）交织成 Thought–Action–Observation 闭环，解决了纯推理易幻觉、纯行动无规划的”两张皮”问题；Reflexion 进一步把失败轨迹压缩为语言反思，实现无需梯度更新的”语言强化学习”。沿工具学习线索，Toolformer 证明模型可以自监督地学会何时调用 API，Gorilla 与 ToolLLM/ToolBench 则把工具规模推进到数千乃至一万六千个真实 API，并给出数据构造、检索增强微调与自动评测的完整闭环。沿环境交互线索，Mind2Web、WebArena、VisualWebArena 建立了 web agent 的离线/在线评测标准，SeeAct 与 WebVoyager 分别代表”文本 grounding”与”纯视觉 Set-of-Mark”两条多模态操作路线；OSWorld 则把评测搬到真实虚拟机，暴露了 computer-use agent 与人类之间曾高达数倍的差距。我们深度剖析 ReAct、ToolLLM、WebArena、OSWorld、UI-TARS 五篇代表作，并追踪 2025–2026 前沿：UI-TARS-2 的”数据飞轮 + 多轮 RL”训练法、GUI-R1 引领的 R1-style 强化后训练浪潮、Agent S2 的 generalist-specialist 组合架构、MCP 协议成为工具接口事实标准及其安全研究、agentic RAG 与自我进化记忆。报告最后给出技术路线对比、开放问题与面向入门研究者的机会点评。全文仅引用经 arXiv 核验的文献，所有关键数字均可在原文复核。

1 引言：本方向在多模态版图中的位置

在此前的专题里，多模态大模型扮演的角色本质上是”观察者”与”评论员”：给它一张图，它描述内容；给它一段视频，它回答问题。这类能力的上限，是一个极其博学的”口述者”——它知道一切，却什么也不能做。而真实世界的任务从来不是静态问答：订一张机票需要跨多个页面填写表单，整理一份报表需要在浏览器、Excel 和文件系统之间来回切换，查一个 bug 需要读代码、跑测试、改文件。完成这些任务，模型必须走出”输入-输出”的一次性映射，进入”感知-决策-行动-再感知”的循环，并在循环中调用自己不具备的能力——计算器、搜索引擎、数据库、代码解释器、成百上千的第三方 API。这就是本专题的主题：**工具使用（tool use）与智能体（agent）**。

有一个形象的概括：如果说视觉-语言模型给了机器”眼睛”，那么 agent 技术教这双眼睛”会用工具”。VLM 提供感知前端，LLM 提供推理中枢，而 agent 框架负责把两者组织成一个能在环境中闭环行动的完整系统。

学习地图（先修概念 → 核心问题 → 关联方向）：

- **先修概念**：自回归语言模型与上下文学习（in-context learning）；思维链提示（chain-of-thought prompting）；VLM 的基本架构（视觉编码器 + connector + LLM，见专题 01）；监督微调（SFT）与基本的强化学习概念（策略、奖励、轨迹）。具备这些即可顺畅阅读全文。
- **本方向的核心问题**：四个层次——（1）**范式**：推理与行动如何交织，才能既规划又纠错？（2）**工具**：模型如何学会选择、调用、组合外部工具，并从工具结果中读取信息？（3）**界面**：模型如何把”点击搜索按钮”这样的抽象决策落到屏幕上的具体坐标（grounding 问题）？（4）**长程**：在几十上百步的任务里，记忆、反思与规划如何对抗误差累积？
- **与其他方向的关系**：本方向站在多条线索的交汇点上。VLM（专题 01）是 GUI agent 的感知基座；具身智能（专题 06）与本方向共享”感知-行动循环”的范式，区别只在动作空间

是机械臂还是鼠标键盘；对齐与安全（专题 09）在 agent 场景下变得尤为尖锐——一个能真实操作你电脑的模型，其误用与失控风险远超聊天机器人；效率（专题 08）则决定 agent 能否以可承受的成本跑完上百步的轨迹。

下图给出多模态模型发展的总体时间线，可以看到 2023 年以后，agent 相关工作（HuggingGPT、WebArena、OSWorld、UI-TARS……）密度显著上升，成为多模态研究的主战场之一：

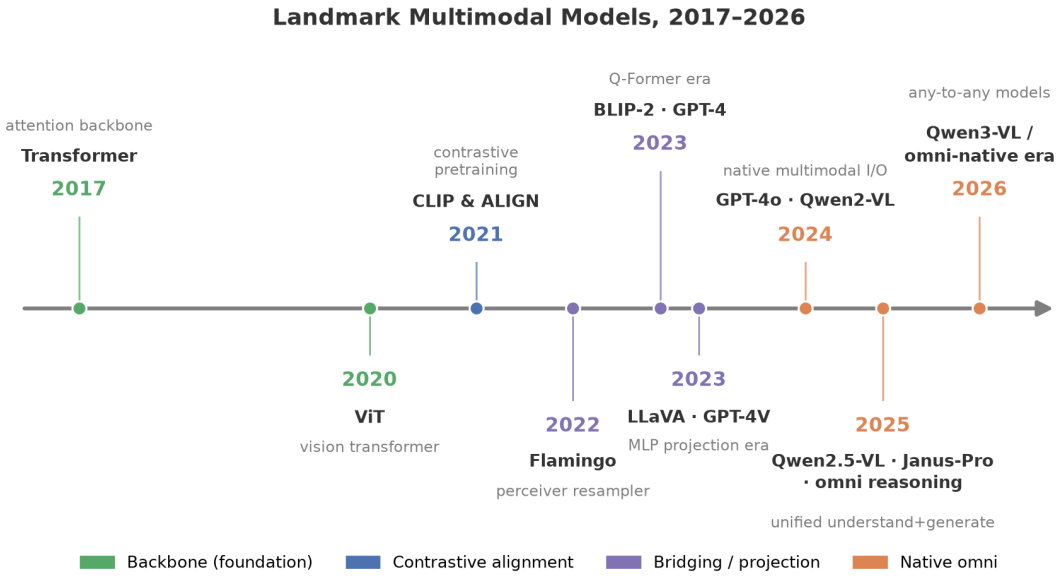


图 1 多模态模型时间线

为什么这个方向在 2023–2025 年爆发？三个条件恰好在此时成熟：其一，LLM 的推理能力越过阈值，思维链提示让”用语言做规划”成为可能；其二，GPT-4V 级别的多模态模型让模型第一次能直接”看懂”屏幕截图，不再依赖 DOM、a11y tree 等内部表示；其三，开源社区积累了足够的微调与强化学习基础设施（PEFT、GRPO、vLLM 等），使学术团队也能训练原生 agent 模型。三条线索的汇合，构成了本报告的全部内容。

2 基础知识

2.1 从一次性映射到决策循环：agent 的形式化

传统 LLM 应用是一个函数 $f : \mathcal{X} \to \mathcal{Y}$，把输入文本映射到输出文本。Agent 则是一个在环境中的序贯决策过程。借用强化学习的语言，可以把 agent 与环境的交互形式化为一个（部分可观测的）马尔可夫决策过程：

- **状态** s_t：环境的真实状态（如整个操作系统的内存状态），agent 通常无法直接观测；
- **观测** o_t：agent 实际看到的东西——一段文本、一张屏幕截图、一棵 DOM 树；
- **动作** a_t：agent 输出的操作——调用一个 API、点击坐标 (x, y)、输入一段文字；
- **转移** $s_{t+1} \sim P(\cdot \mid s_t, a_t)$：环境对动作的响应，agent 不可控；

- **奖励/任务判据** R：任务是否完成，通常由规则脚本或另一个模型判定。

Agent 的策略 $\pi_\theta(a_t \mid h_t)$ 以**历史轨迹** $h_t = (o_1, a_1, \ldots, o_t)$ 为条件。LLM-based agent 的特殊之处在于：h_t 被直接编码进语言模型的上下文窗口，策略就是" 给定全部历史，生成下一步"。这带来两个直接推论：第一，**上下文窗口就是 agent 的工作记忆**，窗口长度与信息组织方式直接决定 agent 能处理的任务跨度——当轨迹超出窗口，早期信息被丢弃，agent 实际上患上了" 失忆症"，这正是各类记忆压缩与外部记忆模块的动机；第二，**误差会沿轨迹累积**——第 t 步的错误动作会污染第 $t+1$ 步的观测与历史，T 步任务的成功率随步数大致呈几何衰减：若单步成功率为 p，全程成功率约 p^T，$p = 0.95$ 时 50 步任务只剩约 8%。理解这两点，就理解了为什么" 记忆管理" 与" 反思纠错" 会成为本方向的核心技术主题。

还值得注意的是与经典 RL 的两点差异。其一，经典 RL 的策略通常从零训练，而 LLM-based agent 的策略**预载了海量的世界知识与程序性常识**——它" 天生" 知道浏览器有地址栏、表单要按提交键，这是零样本泛化的来源，也是这个领域能在三年内爆发的前提。其二，经典 RL 的观测-动作接口是数值化的（状态向量、动作向量），而这里是**语言化的**：观测被序列化为文本/图像 token，动作以文本生成——语言既是推理的媒介也是控制的接口，这一统一是 LLM agent 区别于一切前深度学习时代 agent 架构（BDI、层次任务网络等）的本质特征。

2.2 思维链与" 用语言表达中间状态"

思维链（chain-of-thought, CoT）提示的发现在于：让 LLM 在给出答案前先写出中间推理步骤，可以大幅提升数学与逻辑任务的正确率。对 agent 而言，CoT 的价值不止于正确率——**推理轨迹是 agent 状态的可读外化**。当模型写下" 我已经搜过了甲，结果不相关，接下来应该换关键词搜乙" 时，它实际上在执行三件对 agent 至关重要的事：归纳历史（诱导）、跟踪当前计划（追踪）、并据新信息修正计划（更新）。

但纯 CoT 有一个致命弱点：它是**封闭世界推理**。模型只能依据参数记忆中的知识推理，一旦涉及实时信息（今天的股价）、长尾事实（某小城的邮编）或精确计算（大数乘法），推理链越漂亮，幻觉越自信。CoT 的另一个隐性代价是**错误传播的单向性**：推理链中没有外部反馈，第一步的事实性错误会被后续步骤当作前提继续演绎，错得越远越难以察觉。反过来，纯行动式方法（早期用 RL 或行为克隆训练的 web agent、机器人策略）虽然能接触真实环境，却缺乏显式规划：策略网络把历史压缩进隐状态，面对需要" 先做 A 观察结果、再决定走 B 还是 C" 的条件式任务，容易陷入局部行为模式，失败时也无法向人解释" 它当时在想什么"。

把两条线的失败模式放在一起看，互补结构几乎是自己走出来的：**推理需要行动来落地——把中间结论拿到外部世界验证、取回推理所需的新信息；行动需要推理来导航——在每一步显式地归纳历史、评估进展、修正计划。**这正是 ReAct 的出发点（§4.1）。值得注意的是，这个互补结构在 2024–2025 年经历了第二次表达：推理模型（o1/R1 式）把 CoT 内化进权重，agentic 训练则把工具调用织入推理链，模型在单次生成中交替" 思考段" 与" 工具调用段"——思想内核与 ReAct 一致，只是从提示工程升格为训练目标。

2.3 动作空间设计：agent 的" 手" 长什么样

Agent 的能力上限很大程度上由动作空间决定。本方向常见的动作空间按抽象层级从低到高排列：

1. **坐标级动作**：`click`(x, y)、`type`(text)、`scroll`(dir)。最通用——任何 GUI 都能点，但需要模型具备精确的视觉 grounding 能力。
2. **元素级动作**：`click`(element_id)，其中 element 来自 DOM 节点列表或视觉检测结果。把 grounding 问题部分外包给感知模块，动作更准确，但依赖界面结构信息的质量。
3. **API 级动作**：`search_product`(query) 这样的语义化函数调用。最稳定、最高效，但要求环境预先提供 API——对任意真实网站/软件不成立。
4. **代码级动作**：直接生成一段可执行代码（Python、JavaScript）作为动作。表达能力最强，Voyager（Minecraft）与许多自动化框架采用此路线，代价是执行错误难以恢复。

一个贯穿全报告的张力是：**动作空间越底层越通用，越高层越可靠**。computer-use 的终极目标——像人一样操作任意软件——迫使主流研究走向坐标级动作 + 视觉 grounding 的" 纯视觉" 路线，这也是 GUI grounding 成为兵家必争之地的原因。

2.4 工具调用的三个子问题

" 用工具" 听起来像一件事，拆开却是三个独立的子问题，各自对应一类技术：

- **何时调用（when）**：模型需要判断当前问题是参数记忆可以可靠回答的，还是应该求助外部工具。判断失误的代价是双重的：该调不调导致幻觉，不该调而调浪费成本还可能引入噪声。Toolformer 用" 插入 API 调用是否降低困惑度" 的自监督信号解决此问题（§3）。
- **调用什么（which & how）**：从候选工具集中选出正确的工具并填对参数。当工具数量从 5 个增长到 16000 个，这从提示词问题演变为检索问题——先用检索器召回候选 API，再让模型在候选中决策（Gorilla、ToolLLM 的共用范式）。
- **如何用结果（how to consume）**：工具返回结果后，模型需要正确解析并融入后续推理。看似琐碎，实践中却是失败重灾区：API 返回的冗长 JSON、网页返回的 HTML 噪声，都会稀释模型的注意力。

2.5 Grounding：从语言决策到物理动作

Grounding（落地/对齐）在多模态语境下泛指" 把语言符号绑定到感知实体"。对 GUI agent，它特指：**把模型输出的文本化动作意图（如" 点击' 登录' 按钮"）解析为界面上的具体可执行目标（坐标或元素）**。这是 GUI agent 区别于纯文本 agent 的本质难题，也是 2024 年以前系统的主要瓶颈——SeeAct 的实验（§4.4）显示，GPT-4V 的规划若由人工代为 grounding，在线任务成功率可达 51.1%，而自动 grounding 策略会把这个数字砍掉一大半。规划与落地的差距（planning-grounding gap）是整个方向的头号拦路虎。

主流 grounding 技术路线有三：

- **Set-of-Mark（SoM）**：先用检测器（或可解析 DOM 时直接用布局信息）找出截图中所有可交互元素，在图上为每个元素绘制编号边框，然后把标注后的图喂给 VLM，模型只需回答" 点击 17 号"。这把最难的坐标回归问题转化为选择题，大幅降低了 VLM 的负担，是 WebVoyager 等系统的核心组件。代价是：检测器成为新的误差源，密集界面上编号相互遮挡，且每次观测都要付出一次检测推理的延迟。

- **结构化表示选择**：解析 DOM（网页）或无障碍树（桌面/移动应用），把界面序列化为文本元素列表，让模型在列表中选择目标，再由框架把选中元素映射回坐标。精度最高，但依赖应用对无障碍标准的实现质量——大量真实软件的无障碍树残缺或语义混乱，这也是纯文本观测在桌面端表现远逊于网页端的原因。
- **端到端坐标预测**：模型直接输出 (x, y) 坐标。这需要大规模"截图 → 点击位置"数据训练（CogAgent、UI-TARS 的路线），一旦学成则最为通用——不依赖任何界面内部表示，与人类的操作方式同构。其难点在于高分辨率下的像素级精度：在 1080p 屏幕上点中一个 20×20 像素的按钮，等效于千分之二级的定位精度。

§6 将系统对比三者的工程权衡。这里先记住一个经验结论：**grounding 精度是 computer-use agent 的"乘性瓶颈"**——规划再好，点不中按钮一切都是零；反过来，SeeAct 的人工 grounding 对照实验也表明，把 grounding 补满，当时的 GPT-4V 规划能力已经能撑起过半的在线任务成功率。

2.6 Agent 的评测：为什么"打分"本身是个研究问题

Agent 评测比静态 benchmark 难一个量级，原因有三。其一，**环境是有状态的**：agent 的每个动作都改变环境，评测必须真正执行交互，不能离线比对答案。其二，**成功判据是功能性的**：订到机票的正确答案不是某段文本，而是数据库里多出一条正确订单——这催生了**执行式评测**（execution-based evaluation）：在环境里跑一个脚本，检查最终状态是否满足条件。WebArena、OSWorld 均以此为准。其三，**过程同样重要**：只看成败会低估那些"走对 9 步、错在最后一步"的 agent，WebCanvas 因而提出细粒度中间步骤打分，衡量部分进展。

此外还有第四个困难，往往被忽视：**评判开放终点**。真实任务常有多个合法完成路径（订机票可以直飞可以转机），规则脚本难以穷举，于是出现了 LLM-as-judge 的折中：让强多模态模型根据轨迹截图判断任务是否完成。WebVoyager 报告其 GPT-4V 自动评判与人类判断有 85.3% 的一致率——可用但不完美，且评判模型的偏置（例如倾向于相信"看起来像完成了"的轨迹）会系统地渗入评测结论。

评测的困难也解释了本领域的一个现象：基准的迭代速度几乎与方法一样快，"基准-方法"协同进化（OSWorld → Agent S → UI-TARS → OSWorld 2.0）是推动领域前进的主要机制。对读者而言，这意味着阅读本领域的论文时必须养成一个习惯：**先看评测设置，再看方法创新**——同一个模型在不同观测空间、动作空间、步数预算下的分数可以相差数倍，脱离 harness 谈分数是没有意义的。

2.7 RAG：最朴素的工具使用

检索增强生成（Retrieval-Augmented Generation）可以看作"只会一种工具的 agent"：查询被发给检索器，返回的文档拼进上下文。其动机与工具使用完全同源——参数记忆有限且静态，外部知识库则无限且可更新。多模态 RAG 把这条线推广到图文混合语料：检索器是多模态编码器，语料包含图像、表格、图表，生成器是 VLM。而当检索不再是固定的一步，而变成 agent 规划中的一环——agent 自主决定**是否检索、检索什么、检索几次、何时停止**——就进入了 2025 年兴起的 **agentic RAG** 范畴：RAG 从"管线"变为"策略"。

3 经典方法谱系

本方向的历史虽短，谱系却异常清晰：2022 年 ReAct 立范，2023 年提示工程与工具学习百花齐放，2024 年评测环境成熟、多模态 agent 登场，2025 年原生 agent 模型与强化后训练接棒。以下按四条线索梳理。

3.1 推理-行动框架：从 ReAct 到 Reflexion 到搜索

ReAct（2022，§4.1）确立了基本范式：交错生成 Thought/Action/Observation。它此前，推理（CoT）与行动（RL policy）是两个互不相干的研究传统；它之后，”想一步、做一步、看一步”成为几乎所有 agent 的默认骨架。**Reflexion**（2023）补上了闭环的最后一环——失败之后怎么办。它的关键洞察是：传统 RL 用梯度更新把经验烙进权重，但 LLM 的上下文本身就是一种可写的”记忆介质”，失败轨迹可以被压缩成一段语言反思（”上次我错在没检查文件是否存在就直接覆盖”）存入情景记忆，下次 trial 时读回。这种”语言强化学习”无需任何参数更新，却在 HumanEval 上达到 91% pass@1，超过当时 GPT-4 的 80%（论文报告数字）。Reflexion 的架构由三个角色组成：Actor（产生轨迹）、Evaluator（给轨迹打分，可用环境奖励或模型自评）、Self-Reflection（把分数 + 轨迹转译为语言反思）。这个三角色拆解影响了后来大量”生成-评判-修正”式工作流。它的局限同样值得记取：反思质量依赖模型的自我诊断能力——如果模型根本不知道正确答案长什么样（如开放式研究任务），反思就可能沦为”自信的废话”；且反思是 trial 级的，对单条长轨迹内的中途纠错无能为力。**Tree of Thoughts**（2023）则从另一条路增强规划：既然单链推理会走死，就把推理组织成搜索树，每个节点是一个中间”思维”，配合状态评估（模型自评或投票）与回溯/剪枝——它把”agent 的规划”显式映射为经典 AI 的启发式搜索，思想深刻但推理成本高昂（每个任务数十上百次模型调用），因此更多作为思想资源影响了后续的搜索式推理，而非直接部署的实用系统。

3.2 工具学习：从教模型调 API 到万级 API 生态

2023 年初，Meta 的 **Toolformer** 提出一个优雅的自监督方案：对语料中的每个位置，采样可能的 API 调用插入，执行之，若插入后文本的（加权）困惑度下降则保留为训练样本。模型由此自学”何时调哪个工具、传什么参数”，在算术、事实问答等任务上以 6.7B 规模媲美大得多的模型。Toolformer 证明了工具使用可以**内化进权重**而非外挂于提示，但其工具集是固定的几个。随后两条线解决规模化问题：**Gorilla** 用检索感知微调（retriever-aware fine-tuning）处理 1600+ ML API，检索器负责跟上文档更新，模型只学”根据检索到的文档写调用”；**ToolLLM**（§4.2）更进一步，从 RapidAPI 采集 16464 个真实 RESTful API，用 ChatGPT 自动构造 ToolBench 指令微调数据集，并提出 DFSDT 决策树搜索增强多步推理。这一阶段还有两支值得记住的多模态工具编排工作：**HuggingGPT** 让 LLM 充当控制器，按”任务规划 → 模型选择 → 任务执行 → 响应生成”四阶段调度 HuggingFace 社区上的数百个专家模型（ASR、图像生成、目标检测 ……），把”工具”的概念从 API 推广到整个模型生态；**Visual ChatGPT** 与 **MM-REACT** 则用提示管理器把 20 余个视觉基础模型（OCR、caption、图像编辑）串联在 ChatGPT 周围，使纯文本的 ChatGPT 获得看图、画图、改图的能力。这两条”LLM 调度专家模型”的路线在今天看来有些过渡色彩——端到端 VLM 很快把其中大部分能力内化——但它们确立的”任务分解 + 专家路由”模式，在多 agent 系统中以新的形态延续至今。到 2023 年底，function calling 已成为商用 API 的标准配置，工具学习从研究课题变为工程基础设施。

3.3 Web agent：从静态数据集到真实互联网

Web 是 agent 第一个大规模练兵场。**Mind2Web**（2023）提供 137 个真实网站、2350 个任务的众包操作序列，并发现真实网页 HTML 太长无法直接喂给 LLM，需先用小模型做元素筛选——" 小模型过滤 + 大模型决策" 的两阶段范式影响深远。**WebArena**（2023，§4.3）把评测升级为自托管真实站点的执行式评测，GPT-4 agent 仅 14.41% 的成功率（对照人类 78.24%）给领域泼了第一盆冷水。**SeeAct**（2024）证明 GPT-4V 可以当通用 web agent，但 grounding 是主要瓶颈；**WebVoyager**（2024）用纯截图 + Set-of-Mark 走端到端路线，在 15 个真实网站上取得 59.1% 成功率（GPT-4V 自动评判，与人类判断 85.3% 一致）。值得展开的是 SeeAct 与 WebVoyager 的设计分歧，它预示了后来整个 GUI 领域的路线之争。SeeAct 把 GPT-4V 的规划输出（文本化的" 点击搜索按钮"）通过两类 grounding 策略落地：在 HTML 文本中匹配元素，或用图像标注做元素定位——本质是" 文本决策 + 外部落地"。WebVoyager 则走向另一端：给截图打 Set-of-Mark 编号，让模型直接在视觉空间里做选择与行动——" 视觉决策 + 视觉落地"。前者规划与落地解耦、便于诊断，后者端到端、模块少但依赖标注质量。这个分歧在 computer-use 时代原样重现为" 框架派 vs 原生派"（§5.2）。**VisualWebArena**、**MMInA**（跨站点多跳任务，揭示长链路的复合失败率——每步 90% 成功率，十步任务就只剩三成）、**WebCanvas**（过程打分）分别从视觉必要性、长链路、细粒度三个方向扩展评测。

3.4 GUI / computer-use agent：从外挂管线到原生模型

2024 年是 computer-use 元年。**CogAgent** 以 18B VLM 专攻 GUI 理解与 grounding；**OmniParser** 把" 屏幕解析" 独立成可复用模块（检测可交互元素 + OCR + caption）；**OSWorld**（§4.4）建立真实虚拟机评测标准，彼时最好系统成功率仅 12% 量级（对照人类 72%+）。2024 年 10 月 Anthropic 发布 **Claude computer-use**（官方发布，无 arXiv），首次提供商用" 截图-操作" 闭环 API；2025 年 1 月 OpenAI 发布 **Operator/CUA**（官方发布，无 arXiv），computer-use 进入消费级产品。学术侧的方法论之争同步白热化：**Agent S/S2** 代表" 框架派"——用 generalist 规划器 + specialist grounding 专家 + 经验记忆的组合架构逼近上限；**UI-TARS**（§4.5）代表" 原生派"——把感知、推理、grounding、记忆全部内化进单一端到端 VLM。2025 年 **GUI-R1** 把 R1-style 规则奖励强化学习（GRPO）引入 GUI 领域，仅用 3k 样本（约为可比 SFT 方法数据量的 0.02%）即超越 SFT 基线，宣告 **RL post-training 成为新主战场**；**UI-TARS-2** 以数据飞轮 + 多轮 RL 把 OSWorld 推到 47.5 分。移动端线索（Android in the Wild 的 71.5 万条人类操作数据、AppAgent 的" 自主探索 App 并生成文档再据此操作"、Mobile-Agent-E 的自进化记忆、InfiGUIAgent 的内化推理与反思）与桌面端共享方法栈，主要差异在交互模式（触摸 vs 键鼠）与界面密度（移动端元素更大更规整，grounding 难度略低），本报告不再单列。回看 2024–2025 这两年，computer-use 的进展速度在多模态各方向中几乎无出其右：OSWorld 分数从 12%（2024 年初基线）到 15%（Claude computer-use，2024 年 10 月）到 24.6（UI-TARS，2025 年初）到 35% 量级（Agent S2，2025 年 4 月）再到 47.5（UI-TARS-2，2025 年 9 月）——**十八个月内接近四倍**，而人类基准 72% 仍是前方的灯塔。这个曲线的斜率，比任何单篇论文都更能说明这个方向所处的历史位置。

3.5 关键论文一览表

论文	年份	核心思想	架构/方法要点	备注
ReAct [1]	2022	推理与行动交织	Thought-Action-Observation 循环，few-shot 提示	agent 范式原点，ICLR 2023
Toolformer [8]	2023	自监督学调用工具	API 调用插入 + 困惑度筛选	工具使用内化进权重，NeurIPS 2023
Reflexion [9]	2023	语言强化学习	失败轨迹 → 语言反思 → 情景记忆	HumanEval 91% pass@1，NeurIPS 2023
HuggingGPT [10]	2023	LLM 调度专家模型	任务规划-模型选择-执行-响应四阶段	工具编排代表作，NeurIPS 2023
Tree of Thoughts [12]	2023	推理即搜索	思维树 + 状态评估 + 回溯	长程规划经典基线，NeurIPS 2023
Voyager [13]	2023	终身学习 agent	自动课程 + 代码技能库 + 迭代提示	Minecraft 中自我提升，TMLR 2024
Gorilla [14]	2023	检索感知 API 调用	retriever-aware 微调 LLaMA	1600+ ML API，NeurIPS 2023
ToolLLM [15]	2023	万级真实 API 工具学习	ToolBench + DFSDT + ToolEval	数据-方法-评测闭环，ICLR 2024
Mind2Web [16]	2023	通用 web agent 数据集	小 LM 筛元素 + LLM 决策	137 站 2350 任务，NeurIPS 2023
WebArena [17]	2023	真实站点执行式评测	自托管电商/GitLab/Reddit 等	GPT-4 仅 14.41%，ICLR 2024
AgentBench [19]	2023	多环境统一评测	8 类交互环境横向对比 25+ LLM	ICLR 2024

论文	年份	核心思想	架构/方法要点	备注
Android in the Wild [20]	2023	移动端操作数据	71.5 万条人类 episode	NeurIPS 2023 D&B
SeeAct [21]	2024	GPT-4V 作 web agent	元素标注 + 文本选择 grounding	揭示 planning-grounding gap，ICML 2024
WebVoyager [22]	2024	纯视觉端到端 web agent	截图 + Set-of-Mark + 在线交互	15 站 59.1% 成功率，ACL 2024
VisualWebArena [23]	2024	视觉必需的 web 任务	WebArena 多模态扩展	暴露文本 agent 天花板，ACL 2024
CogAgent [24]	2024	GUI 专用 VLM	高/低分辨率双编码器 18B	VQA-for-GUI 预训练，CVPR 2024
AppAgent [25]	2024	文档化 App 操作	自主探索生成文档再操作	免后端 API，ICLR 2025
OSWorld [27]	2024	真实 OS 评测环境	虚拟机 + 执行式评测 369 任务	computer-use 标准测试床，NeurIPS 2024
OmniParser [29]	2024	屏幕解析模块	元素检测 + OCR + caption	纯视觉 GUI agent 的基础设施
Agent S [31]	2024	经验增强分层框架	分层规划 + Agent-Computer Interface	OSWorld 大幅刷新，ICLR 2025
UI-TARS [32]	2025	原生端到端 GUI agent	感知/推理/grounding/记忆单模型	OSWorld 24.6（50 步），字节跳动
GUI-R1 [37]	2025	R1-style RL 训 GUI	GRPO + 统一动作空间规则奖励	3k 样本超 SFT 基线

论文	年份	核心思想	架构/方法要点	备注
Agent S2 [38]	2025	generalist-specialist 组合	Mixture-of-Grounding + 前瞻分层规划	OSWorld 开源方案长期领先
UI-TARS-2 [39]	2025	数据飞轮 + 多轮 RL	混合 GUI/游戏/SDK 训练 + 统一沙箱	OSWorld 47.5, Android-World 73.3

4 代表作深度剖析

本章精选五篇代表作，覆盖本方向的四个关键断面：范式原点（ReAct）、工具学习（ToolLLM）、web 评测（WebArena）、computer-use 评测（OSWorld）、原生 GUI agent（UI-TARS）。

4.1 ReAct：推理与行动的联姻（2022）

研究问题。2022 年的 LLM 研究存在一个割裂：思维链提示证明 LLM 擅长推理，但推理完全发生在参数记忆的封闭世界里，无法获取新信息，幻觉沿推理链传播；另一边，用 LLM 生成动作计划在具身与游戏环境也有尝试，但这些方法缺乏显式的推理与纠错机制，规划脆弱。两条线各自有致命短板，能否让二者协同？

关键洞察。推理轨迹与行动不是串行两阶段（先想完再做），而应该**逐步交织**：推理帮助模型归纳观察、跟踪并修正计划，行动让模型从外部世界（知识库、环境）取回推理所需的信息。语言和动作是同一个生成过程的两个” 声道”——这一把” 规划” 还原为” 语言建模” 的视角，是 ReAct 最深远的贡献。

技术方法。方法本身简单到可以用一句话说完：在 few-shot 提示中，把任务轨迹写成 $\text{Thought}_1, \text{Act}_1, \text{Obs}_1, \text{T}$ 的交错序列，模型照此格式续写。其中 Thought 是自由文本推理，Act 是预定义动作空间中的动作（如 Wikipedia 环境的 `search[entity]`、`lookup[keyword]`），Obs 是环境返回结果。整个系统就是一个提示模板加一个能执行动作的薄壳——没有训练、没有新架构。作者还探索了 ReAct + CoT 的混合策略（让模型在两种模式间切换）以及对 HotpotQA 的小规模微调。

实验结果。在知识密集型任务（HotpotQA 问答、FEVER 事实核查）上，ReAct 通过与 Wikipedia API 交互克服了纯 CoT 的幻觉与错误传播问题；在交互式决策任务上优势更为显著：ALFWorld（文字版家庭任务）与 WebShop（模拟网购）两个 benchmark 上，ReAct 分别以**绝对 34% 与 10%** 的成功率优势超过当时的模仿学习与强化学习方法（论文报告数字），而 ReAct 只用了一到六个 few-shot 示例。人类评估还显示其轨迹比无推理基线更具可解释性。

批判性分析。优点：范式级贡献，实现成本几乎为零，至今仍是 LangChain 等所有 agent 框架的默认循环；轨迹天然可读，为调试与对齐研究提供便利——当 agent 失败时，你可以直接阅读它的 Thought 序列定位” 它哪一步想错了”，这种可解释性是 RL 策略无法提供的。局限：（1）能力上限被 few-shot 提示锁住——模型只是” 模仿” 轨迹格式，长程任务中推理质量随上下文增长而退化；（2）动作空间需人工设计，不具备迁移性——换一个环境就要重写提示与动作解析器；（3）每步一次 LLM 调用，成本与延迟随步数线性增长，百步任务的账单相当

可观；（4）从今天的视角看，ReAct 的 Thought 是” 事后合理化” 还是” 真实规划” 始终存疑——模型完全可能先决定动作再编造与之相符的思考，这一质疑后来部分被推理模型的训练式思维链回应（思考成为被奖励强化的对象，而非自由文本）。可复现性极佳：纯提示方法，代码与提示模板公开。

一句话评价：★★★★★——以最小的技术改动完成范式革命，本方向一切工作的共同祖先。

4.2 ToolLLM：把工具学习推进到万级 API（2023）

研究问题。2023 年的工具学习研究（Toolformer 等）有两个共同的天花板：工具集是玩具规模（几个到几十个 API），且多为单步调用。而真实世界的 API 生态（如 RapidAPI 平台）有数万个接口，完成一个实际任务往往需要组合多个 API。开源 LLM 能否在真实规模的 API 生态中掌握工具使用?

关键洞察。瓶颈不在模型而在**数据与搜索**：其一，人工标注万级 API 的调用数据不现实，但可以用强模型（ChatGPT）自动构造；其二，单链推理（ReAct 式）在多 API 组合任务上容易一步走错全盘皆输，应把推理升级为**树搜索**，允许模型探索多条 API 调用路径。

技术方法。ToolLLM 是一个” 数据-方法-评测” 三件套：

1. **ToolBench 数据构造**，全自动三阶段：(i) 从 RapidAPI Hub 采集 **16464 个真实 RESTful API，覆盖 49 个类别**；（ii）提示 ChatGPT 生成涉及这些 API 的多样指令，兼顾单工具与多工具场景；（iii）让 ChatGPT 为每条指令搜索可行的 API 调用链作为标注解。
2. **DFSDT（Depth-First Search-based Decision Tree）**：标注与推理共用的决策算法。相比 ReAct 的单链生成，DFSDT 允许模型在每个节点评估多个候选动作、沿深度优先方向展开，失败时回溯到祖先节点换分支——本质是把 Tree of Thoughts 的搜索思想特化到 API 调用场景。
3. **ToolEval 自动评测**：从” 通过率”（是否完成任务）与” 胜率”（与参考解对比）两个维度自动评判工具使用质量。

在 ToolBench 上微调的 LLaMA-7B 称为 ToolLLaMA。实验显示其工具使用能力可比肩 ChatGPT 的教师模型，并展现出对未见 API 的泛化（论文报告结论）。

批判性分析。优点：首次把工具学习的三个环节（数据、方法、评测）做成完整闭环，ToolBench 至今仍是工具学习最常用基准；DFSDT 的” 搜索式推理” 预示了 2024–2025 年推理模型（o1/R1 风格）的方向——把单链生成升级为可回溯的树搜索，用推理时的计算换正确率，这一交易如今已是推理模型的标准做法。局限：（1）数据由 ChatGPT 生成，存在教师模型的分布偏置与” 蒸馏天花板”——ToolLLaMA 学到的是 ChatGPT 的工具使用风格而非从环境反馈中习得，遇到教师也不会的 API 组合时无从泛化；（2）API 为 RESTful 文本接口，输入输出都是干净 JSON，与真实 GUI 操作（像素、坐标、异步状态）之间仍有距离，ToolBench 上训练的能力不能直接迁移到 computer-use 场景；（3）DFSDT 的搜索开销随分支因子增长，在需要实时响应的产品场景难以直接部署；（4）RapidAPI 上相当比例的 API 文档质量参差，”16000+ API” 的数字更多体现规模雄心而非有效覆盖。可复现性好：数据、模型、代码全部开源。

一句话评价：★★★★☆——工具学习方向最完整的系统工程，把该方向从” 技巧演示” 带入” 可规模化的科学”。

4.3 WebArena：给 web agent 一个真实的考场（2023）

研究问题。2023 年之前的 web agent 评测有两类，各有硬伤：静态数据集（如 MiniWoB++ 模拟环境）过于简化，与真实互联网脱节；在线真实网站评测则无法复现（网站随时在变），且任务成败难以自动判定。领域缺一个**既真实又可复现**的评测环境。

关键洞察。用**自托管**（self-hosted）解决真实性与可复现性的矛盾：把真实开源应用（电商、GitLab、Reddit、CMS、地图）部署在沙箱中，环境状态可重置、可快照，任务成败用执行式脚本判定。评测的真实性来自" 真实应用的功能复杂度"，可复现性来自" 环境的完全受控"。

技术方法。环境层面，WebArena 部署四类常见站点的功能完整副本：电商（OneStopShop/Magento）、社交论坛（Reddit）、协同开发（GitLab）、内容管理，外加地图工具与知识库（用户手册），共 **812 个长程任务**（正文报告）。任务设计强调功能性成败判定——每个任务附初始状态配置与评测脚本，检查终态是否满足条件（如"GitLab 中是否出现了指定 issue"）。观察空间为网页 URL、HTML/无障碍树，动作空间为元素级操作（click、type、scroll 等）。基线 agent 集成 ReAct 式" 推理在前" 提示，骨架 LLM 包括 GPT-4、GPT-3.5、Llama 系等。

实验结果。最好的 GPT-4 agent 端到端任务成功率仅 **14.41%**，而人类表现为 **78.24%**——五倍多的差距清楚地宣告：web agent 远未解决。错误分析揭示的典型失败模式包括：在长页面中迷失（找不到目标元素后开始随机游走）、重复无效操作（陷入动作循环，直到步数耗尽）、无法从失败中恢复（一次表单填写错误后不知如何回退）。这三类失败有一个共同根源——**缺乏对工作记忆与进展的显式管理**，直接催生了后来反思机制、记忆模块与分层规划的研究。另一个值得注意的数字是：文本 LLM（GPT-4）与当时的多模态方案差距不大，说明在无障碍树可用的网页场景，视觉信息尚未成为瓶颈——这个结论在 VisualWebArena 的任务上被推翻，提醒读者" 是否需要视觉" 强烈依赖任务分布。

批判性分析。优点：重新定义了 web agent 评测标准，此后所有重要工作（WebVoyager、Operator、UI-TARS-2）都以 WebArena 或其变体报告成绩；自托管范式被 OSWorld 等后来者直接继承。局限：（1）812 个任务的站点类型有限，与真实互联网的多样性仍有距离；（2）无障碍树观察空间把视觉问题旁路了——真实用户面对的是像素而非结构化文本，这正是 VisualWebArena（其多模态扩展）要补的课；（3）环境维护成本高。可复现性极佳：Docker 化部署，完全开源。

一句话评价：★★★★★——一个基准改变一个领域的稀有案例，web agent 研究的"ImageNet 时刻"。

4.4 OSWorld：把考场搬进真实操作系统（2024）

研究问题。WebArena 证明了真实环境评测的价值，但人的数字生活远不止浏览器：文档编辑、文件管理、跨应用工作流（从网页查数据填进电子表格再发邮件）才是生产力的主体。浏览器内的 agent 能力再强，也只覆盖了数字工作的冰山一角。能否构建一个支持**任意应用、任意操作系统**的开放式任务评测环境——agent 面对的就是一台真实的电脑，而不是某个被裁剪过的应用沙盒？

关键洞察。评测环境与技术栈解耦：把评测层架在虚拟机之上，通过标准的虚拟机控制接口（启动、快照、执行命令、读写文件、截屏）实现任务配置与状态检查，从而同一套框架可承载 Ubuntu/Windows/macOS 上的任意真实应用。**执行式评测**——检查环境终态而非比对 agent 输出——是开放式任务可靠评测的唯一可行解。

技术方法。OSWorld 提供统一的虚拟机环境，支持任务设置（如预置文件、打开指定应用）、执行 agent 动作、运行自定义评测脚本。在此之上构建了 **369 个真实计算机任务**，覆盖真实 web 与桌面应用、OS 文件 I/O、跨应用工作流。Agent 的观测主要是屏幕截图（可加 a11y tree），动作是坐标级键鼠操作。基线评测了当时最强的 LLM/VLM（GPT-4V、Claude 等）驱动的 agent。

实验结果。结论堪称残酷：人类能完成 **72.36%** 以上的任务，而当时最好的模型 agent 成功率仅 **12% 出头**（论文报告 12.24%）。失败分析显示，主要短板在 GUI grounding（看不懂/点不准界面元素）与长程规划，且 VLM 并未表现出对纯文本 LLM 的明显优势——多模态输入当时没有转化为操作能力。这个结果与 WebArena 形成镜像：网页场景有干净的无障碍树兜底，视觉增益有限；桌面场景无障碍信息残缺，本应是视觉的主场，2024 年初的 VLM 却接不住——**能力的缺口不在" 看见" 而在" 指向"**，模型能描述屏幕上的按钮，却无法稳定地输出落在按钮内的坐标。这一诊断直接定义了此后两年的研究议程：CogAgent、OmniParser 专攻屏幕解析与 grounding，UI-TARS 用大规模轨迹数据把坐标预测内化，OSWorld 分数也从 12% 一路爬升到 2025 年的 47.5（UI-TARS-2）。

批判性分析。优点：computer-use 领域事实标准，此后两年该方向的全部进展（Claude computer-use 的 ~15%、Agent S2 的 ~35%、UI-TARS-2 的 47.5）都以 OSWorld 分数为度量，可以说"OSWorld 分数曲线就是 computer-use 的进步曲线"；执行式评测脚本设计严谨，可复现性强。局限：（1）评测脚本针对任务终态，对" 过程安全" 不敏感（agent 删错文件但完成任务的案例无法识别，这正是 2026 年 OSWorld-Pro 等过程性评测工作的动机）；（2）369 任务覆盖仍以生产力软件为主，专业软件（CAD、IDE 深度操作）涉及少；（3）虚拟机环境运行成本高，大规模 RL 训练 rollout 开销大。可复现性极佳。

一句话评价：★★★★★——computer-use 的度量衡，没有它这个领域无法谈论" 进展"。

4.5 UI-TARS：原生 GUI agent 模型的崛起（2025）

研究问题。2024 年的 computer-use 系统（Agent S、各种 Claude/GPT-4o 管线）本质上都是" 框架"：一个商用大模型 + 大量专家设计的提示词与工作流 + 外挂的解析/记忆模块。这类系统模块间接口脆弱（解析器的输出格式一变，下游提示全部失效）、错误沿管线传播（感知错一分，规划错十分）、且能力被闭源模型锁死（基座模型一更新，整套提示工程可能作废）。能否训练一个**端到端的原生 agent 模型**，只吃截图、直接输出动作，把感知、推理、grounding、记忆全部内化进权重?

关键洞察。GUI agent 的各项能力不应是外挂模块的拼装，而应是同一模型在**专门构造的数据**上学到的不同行为模式；数据而非架构才是主要瓶颈——需要大规模 GUI 截图理解数据、跨平台统一动作轨迹数据、以及能教会模型" 三思而后行" 的推理轨迹数据。

技术方法。UI-TARS 以 Qwen-VL 系 VLM 为基座，四项核心设计：

1. **增强感知（Enhanced Perception）**：用大规模 GUI 截图数据集训练上下文感知的 UI 元素理解与精确描述（captioning），让模型真正" 看懂" 界面而非泛泛描述。
2. **统一动作建模（Unified Action Modeling）**：把桌面、移动、web 各平台的动作标准化进统一动作空间，用大规模动作轨迹数据训练精确 grounding——这是跨平台泛化的关键。
3. **System-2 推理**：把任务分解、反思思考、里程碑识别等多种推理模式显式训练进多步决策，让模型在行动前进行审慎规划——相当于把 ReAct 的 Thought 从提示词技巧升级为

训练目标。

4. **反思式在线轨迹迭代训练**：让模型在真实环境中跑轨迹，自动收集（尤其是失败的）轨迹回流训练，缓解数据瓶颈——这是” 数据飞轮” 的雏形。

实验结果。UI-TARS 在 10 余个评测感知、grounding 与 GUI 任务执行的 benchmark 上达到 SOTA。标志性数字：OSWorld 上 **24.6（50 步预算）/ 22.7（15 步）**，超过 Claude computer-use 的 22.0 / 14.9；AndroidWorld 上 **46.6**，超过 GPT-4o 的 34.5（均为论文报告数字）。一个端到端开源模型首次在标准 benchmark 上击败精心工程的闭源商用框架。

批判性分析。优点：确立” 原生 agent 模型” 范式，证明了数据驱动的端到端路线可以战胜提示工程驱动的框架路线；统一动作空间与 System-2 推理数据构造被广泛效仿；桌面版开源，推动了整个社区的跟进（UI-TARS-2、各家 CUA 模型均沿此路线）。从方法论史的角度看，UI-TARS 的意义在于它把 ReAct 时代的三个外挂组件——Thought（推理）、grounding（落地）、memory（记忆）——全部变成了**可训练的行为维度**，从而使 agent 能力第一次可以被数据规模与训练算力系统性放大，而不是靠提示工程师的手艺。局限：（1）24.6 的 OSWorld 分数距人类 72% 仍有巨大差距，端到端路线同样受困于长程任务；（2）训练数据构造细节（截图来源、轨迹采集方式、各数据组分配比）披露有限，完全复现困难——这也是所有工业界技术报告的通病；（3）单一模型承担全部职能，在需要专业工具（如终端操作）的场景不如框架路线灵活——UI-TARS-2 后来引入混合 GUI/终端环境正是对此的修正；（4）” 原生” 的代价是每次基座升级都要重跑整套数据管线，模型迭代与数据飞轮深度耦合，灵活性上反而不如可插拔的框架。可复现性中等：模型开源，训练数据与管线部分开放。

一句话评价：★★★★★——GUI agent 的”LLaVA 时刻”，把 computer-use 从提示工程时代带入原生模型时代。

5 研究前沿（2025–2026）

5.1 RL post-training：GUI agent 的新主战场

2025 年最重要的方法论转移，是强化学习后训练从推理模型（DeepSeek-R1 式）扩散到 GUI agent。其动机清晰：SFT 需要海量人工或合成轨迹，且模仿学习只学到” 平均行为”；而 GUI 任务的成败信号天然可程序化判定（任务是否完成、动作是否合法），恰好契合 R1-style 的**规则奖励 + GRPO** 范式。**GUI-R1**（2504.10458）是开路之作：通过统一动作空间的规则建模，仅用 3k 高质量样本（论文称仅为可比方法 13M 数据量的 0.02%）做 GRPO 训练，即在 Windows/Linux/macOS/Android/Web 多平台 benchmark 上超越 SFT 基线。随后 **UI-TARS-2**（2509.02544）展示了这条路的规模化形态：数据飞轮（模型 rollout 产生新数据反哺训练）+ 稳定化多轮 RL 框架 + 混合 GUI/文件系统/终端环境 + 大规模 rollout 统一沙箱，把 OSWorld 推到 47.5、WindowsAgentArena 50.6、AndroidWorld 73.3，并在 15 款游戏套件上取得 59.8 的平均归一化分数，展现出超出 GUI 的通用性。2025 年下半年以来，ZeroGUI（零人工标注在线学习）、SCALECUA、EvoCUA 等工作沿此方向密集出现；学术综述（Agentic RL 综述 2512.16301、GUI-RL 综述 2504.20464）已开始系统整理这波浪潮。**一个值得注意的判断：GUI agent 训练正在复刻 LLM 推理训练 2024 年走过的路——从 SFT 模仿转向 RL 探索，数据效率与上限同时跃升。**支撑这个判断的机制是：GUI 环境的奖励信号（任务完成与否、动作是否合法、坐标是否落在目标元素内）可以完全程序化生成，不依赖人类标注也不依赖 learned

reward model，因此规避了 RLHF 的奖励黑客问题，这正是”verifiable reward” 范式最理想的应用场景之一。但硬币的另一面同样明显：RL 的 rollout 需要在真实或仿真环境中逐步执行，单个 episode 动辄几十步、几分钟，训练吞吐比纯文本 RL 低几个数量级——UI-TARS-2 为此专门构建了统一沙箱平台做大规模并行 rollout。**环境吞吐量，而非算法本身，正在成为这个方向的真正瓶颈**，这也为 §7 的” 训练环境经济学” 埋下伏笔。

5.2 组合架构：generalist 与 specialist 的分工

与端到端路线并行，框架路线也在进化。**Agent S2**（2504.00906）代表 2025 年组合式架构的最高水准：核心思想是把认知职责分配给不同模型——generalist 模型负责规划，专门的 grounding 专家负责定位。两项关键创新：**Mixture-of-Grounding**（根据界面特点动态选择/融合多个 grounding 专家）与**前瞻式分层规划**（在多个时间尺度上随观测动态细化计划）。Agent S2 在 OSWorld 上以相对 18.9%（15 步）与 32.7%（50 步）的优势超过 Claude Computer Use 与 UI-TARS 等基线（论文报告数字），长期位居开源方案榜首。框架派与原生派的关系，类似计算机视觉史上” 特征工程 + 分类器” 与” 端到端深度学习” 之争的历史重演：短期内组合架构凭借可插拔的强者模块领先，长期看端到端模型随数据飞轮与 RL 的复利逐渐反超——UI-TARS-2 的 47.5 vs Agent S2 时代的 ~35 已经显现这个趋势。但公允地说，两条路线正在融合：UI-TARS-2 也引入了终端/SDK 工具（承认纯 GUI 操作的局限），框架系统也越来越多地调用原生训练的 grounding 模型。

5.3 MCP：工具接口的标准化与生态

2024 年 11 月 Anthropic 发布 **Model Context Protocol（MCP）**开放规范，定义了模型与工具/数据源之间的标准化 client-server 接口：工具提供方实现 MCP server（暴露工具、资源、提示模板），agent 应用作为 MCP client 按协议发现与调用。其动机是解决工具集成的 M×N 问题——此前每个 agent 框架都要为每个工具写适配层，MCP 将其标准化为 M+N。2025 年 OpenAI、Google、微软等相继采纳，MCP 成为 agentic workflow 的事实接口标准（官方规范发布，无 arXiv）。学术侧的反应同样迅速：MCPAgentBench（2512.24565）评测 agent 对 MCP 工具的实际使用能力，OSWorld-MCP（2510.24563）把 MCP 工具接入 OS 级评测，MCP-SafetyBench（2512.15163）则聚焦安全——当 agent 可以调用真实工具，恶意 server、提示注入、权限越界都成为现实攻击面，2026 年已有针对 MCP 的工具劫持研究（2609.26761）。对入门研究者的提示：**协议层的评测与安全是 2026 年明确的机会窗口**，竞争者尚少、问题清晰、不需要训练大模型。

5.4 Agentic RAG 与多模态检索

RAG 正在从固定管线演变为 agent 的自主策略。综述（2501.09136）把 agentic RAG 刻画为：agent 根据任务规划决定检索时机、查询改写、多轮检索与来源验证，检索与工具调用、记忆读写统一在 agent 决策循环内。多模态维度上，综述 2502.08826、2504.08748 系统整理了多模态检索器、图文混合语料与引用落地（citation grounding）的技术栈；文档智能方向的 MM-RAG 综述（2510.15253）聚焦长文档、表格、图表的检索增强——这一子方向与企业级知识库需求直接对接，是学术与工业价值密度都很高的地带。对多模态 agent 而言，RAG 的意义还在于它是” 最温和的工具”：在教 agent 操作危险的写操作之前，先教会它可靠地读。

5.5 记忆与长程规划：从外挂模块到自我进化

长程任务的失败大多不是单步失败，而是**记忆失败**：上下文被占满、早期关键信息被挤出、错误经验被重复。2025 年的趋势是把记忆从" 外挂向量库" 升级为**自我进化的结构化记忆**。Mobile-Agent-E（2501.11733）的 tips/shortcuts 机制颇具代表性：agent 在任务结束后自我总结可复用的操作技巧（tips）与跨应用流程捷径（shortcuts），写入持久记忆供后续任务检索——这本质上是 Reflexion 与 Voyager 技能库思想在移动 GUI 场景的现代化身。2026 年的 agent memory 综述（2602.06052）将此类工作系统化为" 记忆的下半场：自我进化记忆"，讨论了记忆的写入时机、组织形式（情景/语义/程序性）与遗忘机制。记忆的另一个维度是**程序性记忆**——不是记住" 发生过什么"，而是记住" 怎么做"。Voyager 的技能库是原型：agent 把验证有效的 Minecraft 操作封装为带描述的代码函数入库，后续按描述检索复用；Mobile-Agent-E 的 shortcuts 是其在移动 GUI 的同构物。程序性记忆的价值在于**复合增长**：技能可以调用技能，agent 的能力随库的规模超线性增长，同时规避了上下文窗口的限制——技能以代码/文本形式存于外部，只在需要时读入。与记忆相伴的是规划层级化：Manager-Worker 式多 agent 架构（Mobile-Agent-E 的 Manager + 4 子 agent、Agent S2 的分层规划）把长任务分解为子目标序列，用结构性约束对抗误差累积——Manager 只看里程碑级别的状态，Worker 处理细节，层级本身就是对上下文长度的压缩。但层级化也引入了新的失败模式：Manager 的错误分解会被 Worker 忠实执行，且跨层级的信息损失（Worker 的丰富观测被压缩成一句话汇报给 Manager）可能造成关键线索丢失。

5.6 评测的下一程：更长、更真、更安全

基准迭代仍在加速。**OSWorld 2.0**（2606.29537）把任务拉长到多应用长链路，直接施压长程规划能力；OpenComputer（2605.19769）与 MacArena（2606.06560，macOS 在线评测）扩展了环境多样性；OSWorld-Pro（2609.24890）引入**过程性评测**——不只问" 任务完成了吗"，还问"agent 在过程中有没有做危险操作"；OSGuard（2606.15034）专注 computer-use 安全。与此同时，7B 级开源 computer-use 模型（Fara-7B、Qwen 系 CUA 模型）正在逼近闭源系统，意味着 computer-use 能力即将" 下沉" 到本地设备——隐私敏感场景（本地操作个人电脑）与端侧部署会成为新的应用形态。另一个值得注意的概念是 **harness（脚手架）**研究的兴起（2609.28919、2605.22166）：同样的模型，不同的动作空间设计、观测组织（截图分辨率、a11y tree 是否拼接、历史轨迹保留几步）、重试与自检机制会带来数倍的性能差异——" 如何给模型搭脚手架" 本身成为一门学问，也为评测公平性提出新课题：两个系统报出同样的 OSWorld 分数，可能一个靠模型能力、一个靠精巧的 harness，这对" 模型能力是否进步" 的科学判断构成干扰。社区正在形成的应对是双轨报告：既报" 裸模型 + 标准 harness" 的可比分数，也报" 最佳系统配置" 的上限分数。

6 技术路线对比与权衡

6.1 观测表示：截图、DOM，还是两者？

观测方案	代表工作	优点	缺点	适用场景
纯文本（DOM/HTML）	Mind2Web 基线	信息精确、token 高效	丢失视觉布局；canvas/图片内容不可见	结构规整的网页任务
无障碍树（a11y tree）	WebArena、OSWorld 基线	语义化、比原始 HTML 干净	依赖应用实现质量，桌面端覆盖参差	有良好无障碍支持的应用
Set-of-Mark 截图	WebVoyager、SeeAct	视觉信息完整；grounding 简化为”选编号”	标注遮挡细节；密集界面编号拥挤；每步需检测器	在线 web 操作、快速原型
原始截图（端到端）	UI-TARS、CogAgent	最通用，与人所见一致；无模块级联误差	需要大规模 grounding 数据训练；坐标精度要求高	原生 agent 模型、跨平台泛化
截图 + 结构信息混合	Agent S/S2、OmniParser 管线	互为冗余、鲁棒性最好	系统复杂、上下文成本高	追求成功率上限的框架系统

分析。历史趋势清晰：2023 年文本路线主导，2024 年 Set-of-Mark 折中方案流行，2025 年纯视觉端到端随原生模型成熟成为主流。但”纯视觉胜利”的结论要谨慎——目前 SOTA 系统（UI-TARS-2、Agent S2）在可用时都会利用结构信息或终端工具。真实的共识是：**视觉是必要的（很多信息只在像素里），结构信息是宝贵的冗余（能拿到就别扔）**。

6.2 工具获取方式：提示、微调，还是强化学习？

路线	代表工作	数据需求	能力上限	成本特征
提示工程（ReAct 式）	ReAct、Reflexion、WebVoyager	几个示例	受基座模型限制	零训练成本，推理成本高
SFT（轨迹模仿）	ToolLLaMA、UI-TARS 首阶段	数千到千万级轨迹	模仿数据分布的平均行为	数据构造是主要成本
检索增强微调	Gorilla	API 文档 + 合成调用	可随文档更新扩展	检索器维护成本
RL post-training	GUI-R1、UI-TARS-2	数千任务 + 可判定奖励	可超越模仿数据上限	环境 rollout 成本巨大

分析。四条路线不是替代关系而是栈式组合：当前最佳实践（UI-TARS-2 范式）是”SFT 冷启动建立基本行为 → 数据飞轮扩充覆盖 → 多轮 RL 突破上限”。纯提示路线在研究原型与小团队中仍有价值（零训练成本、即插即用、基座升级免费获益），但其与训练路线的成功率差距已从 2024 年的” 接近” 拉大到 2025 年的” 代差”——UI-TARS 以单一开源模型击败 Claude 框架是这条分水岭的标志性事件。对算力有限的团队，GUI-R1 的经验最有启发性：**精心设计的规则奖励 + 少量高质量任务**可以撬动远超数据规模的收益——3k 样本超越 13M 样本的 SFT，说明在可验证奖励的场景下，” 让模型自己探索并从成败中学习” 的数据效率可以比” 模仿教师轨迹” 高三个数量级。但也要注意这条路线的隐性门槛：奖励函数的设计、环境 rollout 的基础设施、GRPO 训练的稳定性调参，每一步都有工程陷阱，”3k 样本” 的叙事不应掩盖其背后的系统投入。

6.3 系统形态：单一原生模型 vs 多模块框架

形态	代表	优势	风险
原生端到端模型	UI-TARS 系、CogAgent	无级联误差；能力随训练复利增长；部署简单	训练成本高；能力黑箱难以定向修复
组合式框架	Agent S2、Mobile-Agent-E	模块可替换（新模型即插即用）；行为可审计、可干预	模块接口脆弱；错误沿管线传播；工程复杂度高
层级多 agent	HuggingGPT、Manager-Worker 系	任务分解天然；子 agent 可专业化	协调开销；信用分配困难；成本成倍

分析。短期（2025–2026）的现实答案是混合：原生模型负责” 感知 +grounding” 这类可从数据规模获益的能力，框架负责” 工具编排 + 安全检查 + 成本控制” 这类需要显式控制的职能。长期看，随着 RL 训练环境成熟，原生模型的份额会持续扩大。这个判断的依据来自深度学习史的反复重演：每当” 人工设计的模块化管线” 与” 数据驱动的端到端模型” 竞争，只要（a）数据可以规模化获取、（b）评测信号可以程序化生成，后者最终都会胜出——语音识别、机器翻译、目标检测无一例外。GUI agent 恰好两个条件都在成熟：数据飞轮解决（a），执行式评测解决（b）。但框架不会消亡，它会退居” 安全与治理层”：正如今天没有人用纯神经网络做支付系统的最终闸口，未来 agent 的高危操作大概率仍需框架级的显式授权与审计，这与模型智能程度无关，而是责任结构的要求。

6.4 评测维度权衡

执行式评测（OSWorld/WebArena）保证可复现但任务设计昂贵；在线真实网站评测（WebVoyager 基准、Online-Mind2Web）真实性最高但随网站改版而漂移；LLM-as-judge（WebVoyager 的 GPT-4V 评判，与人类 85.3% 一致）降低评判成本但引入评判模型偏置。**实践建议**：方法开发期用执行式基准做主要信号，辅以在线评测验证泛化，报告时明确标注评测版本与日期——这在网站持续演化的世界里是基本的学术诚实。

7 开放问题与未来方向

1. **长程任务的可靠性工程**。OSWorld 2.0 式多应用长链路任务上，即便最好的 agent 成功率也远低于短任务。单步准确率 95% 的 agent 走 50 步，全程成功率不足 8%——这是几何衰减的数学必然，不是靠把单步准确率从 95% 提到 97% 能解决的（即使 97%，50 步也只剩约 22%）。突破口可能不在更聪明的单步决策，而在**检查点、回滚与验证机制**：让 agent 在关键节点自我验证中间状态、失败时回退到最近检查点而非从头再来——像软件工程对待分布式系统那样对待 agent 轨迹，用冗余与确认换取整体可靠性。这一方向几乎未被现有基准覆盖（它们大多只评终态），是方法论与评测可以共同创新的地带。
2. **Grounding 的最后一公里**。元素密集界面（IDE、CAD、交易终端）、动态渲染内容（canvas、游戏画面）上的精确操作仍未解决。Mixture-of-Grounding 与原生高分辨率编码谁能胜出，是 2026 年的看点。
3. **安全与可控**。能操作真实电脑的 agent 把提示注入从” 说错话” 升级为” 删库跑路”。MCP 生态的攻击面（恶意工具描述、返回内容注入）、过程性安全评测（OSGuard、OSWorld-Pro）、权限最小化设计，都是刚开题的方向。
4. **记忆的可扩展性**。自我进化记忆如何避免” 错误经验的复利”？记忆的写入、检索、遗忘的自动化评判标准尚不存在——这既是系统问题也是评测问题。
5. **训练环境的经济学**。RL post-training 受限于 rollout 成本：真实虚拟机环境昂贵且慢。高保真模拟器、环境并行化、离线 RL 与在线 RL 的配比，是决定学术团队能否参与下一轮竞争的关键基础设施问题。
6. **跨任务与跨平台的正向迁移**。UI-TARS-2 已显示 GUI/游戏/终端混合训练的互益迹象，但迁移的机制（共享的是视觉表征还是动作先验?）未被理解。负迁移的边界在哪里，也缺乏系统研究。
7. **Agent 协作与协议**。MCP 解决了模型 ↔ 工具接口，agent↔agent 协作（A2A 等）尚无公认标准；多 agent 系统的信用分配与失败归因在理论上也不成熟。
8. **评测与现实的鸿沟**。现有基准任务仍以” 明确指令” 为主，真实用户的需求常常模糊、多目标、需澄清。**会提问的 agent**（主动澄清需求的交互式 agent）几乎没有被评测覆盖。

8 研究机会点评（给入门研究者的建议）

算力门槛适中、问题清晰的方向：

- **MCP 生态的评测与安全**（§5.3）：不需要训练大模型，需要的是精巧的攻击构造与评测设计。MCP-SafetyBench、工具劫持研究刚起步，一篇扎实的分析性工作即可立足。
- **Agent 记忆机制**：记忆读写策略可以用冻结的商用 API + 精巧框架完成，算力需求低，问题（写入时机、遗忘、错误经验隔离）清晰且有体系化空间。
- **过程性评测与过程奖励**：OSWorld-Pro 方向刚开题；过程奖励模型（PRM）在 GUI agent 上的迁移几乎是空白。适合有评测设计经验或标注资源的团队。
- **小模型 computer-use**：Fara-7B 证明 7B 级可行。端侧 agent 的隐私叙事强、蒸馏/量化技术栈成熟，单卡到八卡即可参与。
- **Agentic RAG 的检索决策**：把” 是否检索/何时停止” 形式化为可学习的策略，问题定义

干净，实验成本低。

中等算力、高风险高回报：

- **RL post-training for GUI**（GUI-R1 路线）：GRPO 训练 7B 模型需要数十卡时级投入，但问题处于风口；关键是构造可自动判定的任务集——这本身就是可发表的工作。
- **Grounding 专项**：混合专家 grounding、高分辨率编码，需要训练但规模可控。

坦诚地说属于巨头游戏的方向：

- **端到端原生 computer-use 大模型**（UI-TARS-2 级别）：数据飞轮 + 大规模沙箱 rollout 的基础设施投入超出绝大多数学术团队的能力边界。学术团队更现实的策略是在其开源模型上做后训练、分析与评测。
- **全栈商用 agent 产品**：Operator/Claude 级产品需要模型、基础设施、产品三位一体，非学术赛道。

给初学者的路径建议：先用 WebArena/OSWorld 跑通一个 ReAct 式基线（纯 API 调用，零训练），亲手感受 agent 的失败模式——你会亲眼看到它如何在长页面中迷路、如何把同一个按钮点十遍、如何在失败后编造”任务已完成”的总结，这些第一手经验比读十篇论文更能建立直觉。然后精读 ReAct → OSWorld → UI-TARS 三篇，建立”范式-评测-方法”的完整坐标系；再从上述低门槛方向中选一个具体切口。切记：**这个领域最稀缺的不是新点子，而是可靠的实验**——环境跑不通、评测有漏洞的工作每天都在被撤稿或忽略；反过来，一个扎实的失败分析或评测审计，往往比又一个”+2% 的新方法”更受社区尊重。

9 术语表

- **Agent（智能体）**：在环境中循环执行”感知-决策-行动”以完成目标的系统，本报告中特指以 LLM/VLM 为决策核心的系统。
- **ReAct**：Reasoning + Acting 范式，交错生成思考、动作、观察的 agent 基本循环。
- **Thought–Action–Observation（TAO）循环**：ReAct 轨迹的三元结构：推理文本、环境动作、环境反馈。
- **Chain-of-Thought（CoT，思维链）**：让模型先输出中间推理步骤再给答案的提示技术。
- **Reflexion（反思）**：把失败轨迹压缩为语言总结存入记忆、供后续尝试读取的无梯度学习机制。
- **Tool use / Function calling（工具使用/函数调用）**：模型生成结构化请求调用外部 API/工具并消费其结果的能力。
- **ToolBench / DFSDT**：ToolLLM 提出的万级 API 工具学习数据集与其配套的深度优先决策树搜索算法。
- **Grounding（落地）**：把模型的文本化动作意图解析为界面上具体目标（坐标或元素）的过程；也泛指语言符号与感知实体的绑定。

- **Set-of-Mark（SoM）**：在截图上为可交互元素绘制编号框、让模型按编号选择的 grounding 技术。
- **Accessibility tree（a11y tree，无障碍树）**：操作系统/浏览器为辅助技术提供的界面语义结构，常被 agent 用作观测。
- **Computer-use agent（CUA）**：以截图 + 键鼠动作直接操作通用计算机的 agent。
- **Execution-based evaluation（执行式评测）**：通过运行脚本检查环境终态来判定任务成败的评测方式。
- **MCP（Model Context Protocol）**：Anthropic 提出的模型与工具/数据源之间的标准化 client-server 开放协议。
- **Agentic RAG**：把检索增强生成的检索时机、查询构造、轮次控制交给 agent 自主决策的范式。
- **GRPO（Group Relative Policy Optimization）**：以组内相对优势替代价值网络的策略优化算法，R1-style 规则奖励训练的常用算法。
- **数据飞轮（data flywheel）**：模型在环境中 rollout 产生新轨迹、筛选后回流训练的闭环数据机制。
- **Harness（脚手架）**：包裹模型的工程层——观测组织、动作空间、重试与检查机制的总称。
- **OSWorld / WebArena / AndroidWorld**：分别在真实操作系统、自托管网站、Android 模拟器上的执行式 agent 评测基准。

10 参考文献

综述类

1. A Survey on Large Language Model based Autonomous Agents. Wang et al., 2023. arXiv:2308.11432. https://arxiv.org/abs/2308.11432
2. The Rise and Potential of Large Language Model Based Agents: A Survey. Xi et al., 2023. arXiv:2309.07864. https://arxiv.org/abs/2309.07864
3. Agent AI: Surveying the Horizons of Multimodal Interaction. Microsoft et al., 2024. arXiv:2401.03568. https://arxiv.org/abs/2401.03568
4. Tool Learning with Foundation Models. Qin et al., 2023. arXiv:2304.08354. https://arxiv.org/abs/2304.08354
5. GUI Agents: A Survey. 2024. arXiv:2412.13501. https://arxiv.org/abs/2412.13501
6. A Survey on GUI Agents with Foundation Models Enhanced by Reinforcement Learning. 2025. arXiv:2504.20464. https://arxiv.org/abs/2504.20464
7. OS Agents: A Survey on MLLM-based Agents for General Computing Devices Use. 2025. arXiv:2508.04482. https://arxiv.org/abs/2508.04482

推理-行动框架与工具学习

1. ReAct: Synergizing Reasoning and Acting in Language Models. Yao et al., 2022. arXiv:2210.03629. https://arxiv.org/abs/2210.03629
2. Reflexion: Language Agents with Verbal Reinforcement Learning. Shinn et al., 2023. arXiv:2303.11366. https://arxiv.org/abs/2303.11366
3. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. Shen et al., 2023. arXiv:2303.17580. https://arxiv.org/abs/2303.17580
4. Toolformer: Language Models Can Teach Themselves to Use Tools. Schick et al., 2023. arXiv:2302.04761. https://arxiv.org/abs/2302.04761
5. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. Yao et al., 2023. arXiv:2305.10601. https://arxiv.org/abs/2305.10601
6. Voyager: An Open-Ended Embodied Agent with Large Language Models. Wang et al., 2023. arXiv:2305.16291. https://arxiv.org/abs/2305.16291
7. Gorilla: Large Language Model Connected with Massive APIs. Patil et al., 2023. arXiv:2305.15334. https://arxiv.org/abs/2305.15334
8. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. Qin et al., 2023. arXiv:2307.16789. https://arxiv.org/abs/2307.16789

Web agent

1. Mind2Web: Towards a Generalist Agent for the Web. Deng et al., 2023. arXiv:2306.06070. https://arxiv.org/abs/2306.06070
2. WebArena: A Realistic Web Environment for Building Autonomous Agents. Zhou et al., 2023. arXiv:2307.13854. https://arxiv.org/abs/2307.13854
3. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. Koh et al., 2024. arXiv:2401.13649. https://arxiv.org/abs/2401.13649
4. AgentBench: Evaluating LLMs as Agents. Liu et al., 2023. arXiv:2308.03688. https://arxiv.org/abs/2308.03688
5. Android in the Wild: A Large-Scale Dataset for Android Device Control. Rawles et al., 2023. arXiv:2307.10088. https://arxiv.org/abs/2307.10088
6. GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct). Zheng et al., 2024. arXiv:2401.01614. https://arxiv.org/abs/2401.01614
7. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. He et al., 2024. arXiv:2401.13919. https://arxiv.org/abs/2401.13919
8. MMInA: Benchmarking Multihop Multimodal Internet Agents. 2024. arXiv:2404.09992. https://arxiv.org/abs/2404.09992
9. WebCanvas: Benchmarking Web Agents in Online Environments. 2024. arXiv:2406.12373. https://arxiv.org/abs/2406.12373

GUI / computer-use agent

1. CogAgent: A Visual Language Model for GUI Agents. Hong et al., 2023. arXiv:2312.08914. https://arxiv.org/abs/2312.08914
2. AppAgent: Multimodal Agents as Smartphone Users. Yang et al., 2023. arXiv:2312.13771. https://arxiv.org/abs/2312.13771
3. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. Xie et al., 2024. arXiv:2404.07972. https://arxiv.org/abs/2404.07972
4. OmniParser for Pure Vision Based GUI Agent. Lu et al., 2024. arXiv:2408.00203. https://arxiv.org/abs/2408.00203
5. OmniParser V2: Structured-Points-of-Thought for Unified Visual Text Parsing. 2025. arXiv:2502.16161. https://arxiv.org/abs/2502.16161
6. Aguvis: Unified Pure Vision Agents for Autonomous GUI Interaction. 2024. arXiv:2412.04454. https://arxiv.org/abs/2412.04454
7. Agent S: An Open Agentic Framework that Uses Computers Like a Human. Agashe et al., 2024. arXiv:2410.08164. https://arxiv.org/abs/2410.08164
8. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. ByteDance, 2025. arXiv:2501.12326. https://arxiv.org/abs/2501.12326
9. Mobile-Agent-E: Self-Evolving Mobile Assistant for Complex Tasks. 2025. arXiv:2501.11733. https://arxiv.org/abs/2501.11733
10. InfiGUIAgent: A Multimodal Generalist GUI Agent with Native Reasoning and Reflection. 2025. arXiv:2501.04575. https://arxiv.org/abs/2501.04575
11. Claude computer-use. Anthropic, 2024-10. [官方发布/技术报告，无 arXiv]
12. OpenAI Operator / Computer-Using Agent (CUA). OpenAI, 2025-01. [官方发布/技术报告，无 arXiv]
13. GUI-R1: A Generalist R1-Style Vision-Language Action Model for GUI Agents. 2025. arXiv:2504.10458. https://arxiv.org/abs/2504.10458
14. Agent S2: A Compositional Generalist Specialist Framework for Computer Use Agents. Agashe et al., 2025. arXiv:2504.00906. https://arxiv.org/abs/2504.00906
15. UI-TARS-2 Technical Report. ByteDance, 2025. arXiv:2509.02544. https://arxiv.org/abs/2509.02544

协议、RAG 与前沿（2025–2026）

1. Model Context Protocol (MCP). Anthropic, 2024-11 发布规范，2025 年生态爆发. [官方规范，无 arXiv]
2. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. 2025. arXiv:2501.09136. https://arxiv.org/abs/2501.09136
3. MCPAgentBench. 2025. arXiv:2512.24565. https://arxiv.org/abs/2512.24565
4. MCP-SafetyBench. 2025. arXiv:2512.15163. https://arxiv.org/abs/2512.15163
5. OSWorld-MCP. 2025. arXiv:2510.24563. https://arxiv.org/abs/2510.24563

6. ZeroGUI. 2025. arXiv:2505.23762. https://arxiv.org/abs/2505.23762
7. Agentic Reinforcement Learning 综述. 2025. arXiv:2512.16301. https://arxiv.org/abs/2512.16301
8. OSWorld 2.0. 2026. arXiv:2606.29537. https://arxiv.org/abs/2606.29537
9. MacArena. 2026. arXiv:2606.06560. https://arxiv.org/abs/2606.06560
10. Fara-7B. 2025. arXiv:2511.19663. https://arxiv.org/abs/2511.19663
11. Agent Memory 综述：The Second Half of Agent Memory. 2026. arXiv:2602.06052. https://arxiv.org/abs/2602.06052